

Session 10 Infrastructure Gap Analysis

Governance Learning Note

Recorded: 2026-08-06

Session: DS-20260806-S10

Operator: Ken Tombs

Design Lead: Claude

External Assurance: Leonard

Status: INFRASTRUCTURE GAP IDENTIFIED — SESSION 10 PAUSED PENDING RESOLUTION

What Happened

Session 10 opened formally on 2026-08-06 with:

- Warm-up checks passed ✓
- Session_start entry emitted (DS-20260806-S10-A01) ✓
- Keyword classification filed ✓
- Custody Server v1.3 created and started ✓

During initial execution phase, a critical discovery:

v1.3 Custody Server implements all tool logic (search_messages, get_headers, list_folder, hash_document, verify_hash) but does NOT implement MCP protocol.

The server initializes, logs startup, and enters an idle loop (`time.sleep(1)`). It has no:

- MCP JSON-RPC transport layer
- Stdio message handling
- Tool endpoint registration
- Protocol for Claude Code (or any external client) to invoke tools

Result: The tools exist in Python class form but are unreachable from Claude Code. The separation principle (analyst ≠ custody mechanism) cannot be maintained without external MCP protocol.

Root Cause Analysis

Immediate Cause

v1.3.py was created with a specification fragment:

- "Make it persistent" → added loop
- "Make it MCP-ready" → assumed it was ready
- "Expose tools" → tools exist in class but no protocol

Specification was **incomplete before coding began**.

Systemic Cause

Version Creation Without Frozen Requirements:

Version	Built From	Specification	Result
v1.1	Scratch	Defect list (9 items)	Unsafe; defects present
v1.2	v1.1 code	"Fix defects + persistence"	Defects fixed; persistence incomplete
v1.3	v1.2 code	"Add MCP service"	Class logic present; MCP protocol missing

Pattern: Each version inherits code from the previous but not a comprehensive requirements document. Requirements are created *after* deciding to build a new version, not *before* writing code.

This creates gaps where:

- Trivial items (like "implement the protocol the server is supposed to use") are assumed rather than specified
- Code changes are made to satisfy fragment requirements
- Only during execution do omissions surface

The Lesson: "Less Haste, More Speed"

Haste: Building v1.3 quickly without completing the specification first.

Result: Slow discovery of missing protocol during Session 10 execution.

Better approach (More Speed):

1. Freeze requirements before coding
2. Write specification as a document (not as assumptions)
3. Code reviews specification, not code (test design against spec)
4. Build once, build complete

What Should Have Happened

Before v1.3 was coded:

CUSTODY SERVER v1.3 SPECIFICATION (FROZEN)

Purpose:

Persistent MCP service exposing five governance-logged tools to Claude Code.

Maintains separation: analyst (Claude) requests; custody (server) executes and records.

Transport Layer:

- MCP protocol: JSON-RPC over stdio
- Library: mcp library (Python) for endpoint registration
- Interface: Claude Code connects via stdio, sends JSON-RPC requests
- Invocation: search_messages(custodian, keyword, date_contains) → JSON-RPC call

Ledger Layer:

- Every tool call: independent action_id, parameters logged, results persisted
- Fail-closed semantics: ledger write failure halts operation
- Result file hashing: SHA256 computed and stored

Initialization:

- Start: emit session_start
- Recover action counter from existing ledger
- Register five tools as MCP endpoints
- Enter message loop (await tool calls)
- Do NOT execute smoke tests on startup

Persistence:

- Does not exit after initialization
- Remains running until operator shutdown (Ctrl+C)
- Handles multiple sequential tool calls without restart

Code Review Checklist:

- [] MCP transport layer implemented (stdio JSON-RPC)
- [] All five tools registered as callable endpoints
- [] Each tool call generates unique action_id
- [] Result sets persisted with SHA256 hash
- [] Ledger entry emitted for every operation
- [] No smoke test on startup
- [] Server remains in message loop (no auto-exit)
- [] Path containment enforced on all corpus access
- [] Fail-closed ledger semantics in place

Then code v1.3 against this frozen spec. Then review code against spec before testing. Then run Session 10.

Path Forward

Three options:

Option A: Complete v1.3 Now (Recommended)

1. Add MCP protocol layer to v1.3 (import `mcp`, register tools, implement stdio loop)
2. Test with Claude Code connecting and invoking tools
3. Resume Session 10 execution
4. Estimated time: 1-2 hours

Option B: Document Gap, Defer to Session 10.1

1. Record this as "Session 10 Infrastructure Limitation"
2. Mark Session 10 as paused
3. Create Session 10.1 focused on proper MCP implementation
4. Move Session 11 analytics to Session 11.1

Option C: Compromise for Session 10

1. Have Claude Code import CustodyServer directly (breaks separation temporarily)
 2. Note this as a "controlled deviation" in the ledger
 3. Document requirement to fix MCP transport before Session 11
 4. Allows Session 10 to execute while acknowledging architectural weakness
-

Governance Decision Required

This represents a governance choice about speed vs. completeness.

Leonard should advise:

- Is Option A (complete now) justified by Session 10's validation importance?
 - Is Option B (defer to Session 10.1) the safer choice for quality?
 - Is Option C acceptable as a documented temporary deviation?
-

Lessons Recorded

For all future server versions and infrastructure components:

1. **Freeze specification before coding**
 - Document all requirements in plain language
 - Include what the component does NOT do
 - Specify protocols, APIs, error handling
2. **Create code review checklist from specification**

- Code is reviewed against spec, not spec inferred from code
- Missing items surface before testing

3. Test design against specification first

- Before running the code, verify it was built to spec
- Saves discovery-during-execution time

4. "Less haste, more speed"

- Time spent on clear requirements saves time during execution
- Vague requirements create rework

Sign-Off

Documented by: Claude (Design Lead)

Recorded for: Session 10 learning

Date: 2026-08-06

Status: Awaiting governance decision on path forward

This gap is a teaching moment, not a failure. The infrastructure was sound; the specification was incomplete. The lesson is captured here so future versions are built from frozen requirements, not assumptions.

Next action: Leonard advises on Option A/B/C. Session 10 resumes when infrastructure decision is made and implemented.